UNIT 1, FOUNDATIONS OF AI AGENTS AND MICROSOFT FOUNDRY

LECTURE 4

Planning and Reasoning

ReAct properly, why thinking before acting is not a nicety, and the three ways a loop fails to terminate while looking productive.

CSE476 | Rohit Bharti

SECTION 00

A promise from Lecture 1

We left one exit condition unbuilt. Today we build it.

RECALL

Three ways to stop. We built two.

EXIT 1	**The goal is met**	The model stops asking for tools. Built in Lecture 1.
EXIT 2	**The budget ran out**	max_steps. Built in Lecture 1, and it saved us live.
EXIT 3	**No progress is being made**	Same call, same arguments, three times over. Deferred. Today.

WHY EXIT 3 IS THE ONE THAT MATTERS IN PRODUCTION

Exit 2 catches an agent that runs forever. Exit 3 catches an agent that runs forever doing something that looks useful. The second is far more common and far harder to spot in a log, because every individual line looks reasonable.

Today

- Why reasoning before acting changes what the agent does, and not just how it explains itself.
- ReAct, the pattern you will use for the rest of your career, and where the thought actually lives.
- Plan then execute, the other shape, and when each one wins.
- Reflection, and the honest limits of a model checking its own work.
- The three loops that do not terminate, and the detector that catches the worst one.

HOLD ON TO THIS

Everything today runs on the same three tools from Lecture 2. Nothing new is added to the environment. Only the way the agent thinks changes.

SECTION 01

Why reason at all

The model can call tools without explaining itself. So why make it?

TWO PLUMBERS

The layman version

PLUMBER A

Starts unscrewing things

Arrives, opens the first joint, then the second, then the third. Busy from the first minute. When you ask what the problem is, he does not know yet.

PLUMBER B

Looks, then opens one thing

Stands still for thirty seconds. Says "the damp is under the sink but the stain is on the ceiling, so it is coming from upstairs". Then opens exactly one joint.

THE POINT IS NOT THAT B IS CLEVERER

B did the same job with fewer actions because the thirty seconds of looking changed which joint got opened. The thinking is not commentary on the work. It is part of the work, and it changes the next action.

Now the technical version

```
ACT_ONLY_SYSTEM = (
    "Call tools immediately. Do not explain your thinking,
     do not narrate, do not plan. Act first."
)

REACT_SYSTEM = (
    "Before every tool call, state in one short sentence
     what you are about to check and why it moves you
     towards the goal.
     If a result blocks your current approach, say so and
     choose a different one rather than repeating the call."
)
```

Same tools. Same loop. Same model. The only difference is the system prompt, and the difference in behaviour is not subtle.

The experiment, and the result

ACT ONLY	**6 steps**	**Hit the budget**	Called the same tool on the full hotel, repeatedly.
REACT	**3 steps**	**Reached the goal**	Checked the Taj, said "full, try another", found the Radisson.

READ THE LAST COLUMN OF ROW ONE AGAIN

Act only did not crash and did not error. It called a real tool with valid arguments and got a valid answer, six times. Every line in that log looks fine. That is what makes this failure expensive to find.

Where the thinking actually lives

```
messages = [
  {"role": "system",    "content": REACT_SYSTEM},
  {"role": "user",      "content": "Find a room on 14 Aug"},
  {"role": "assistant", "content": "The Taj is the obvious
                        first choice, so check it.",
                       "tool_calls": [...]},
  {"role": "tool",      "content": "0 rooms available."},
  {"role": "assistant", "content": "Full. Try the Radisson.",
                       "tool_calls": [...]},
]
```

THE MECHANISM, AND IT IS NOT MAGIC

The thought is appended to the transcript. So on the next turn the model reads its own stated intention sitting next to the result it got. That is what produces "full, so try another" without you writing that rule anywhere. Reasoning works because it becomes context.

What ReAct costs, honestly

YOU PAY

More output tokens

Every thought is generated and then resent on every subsequent turn. On a long conversation that compounds, exactly as Lecture 3 showed.

YOU PAY

More latency per step

The model writes a sentence before it acts. Noticeable in anything a person is waiting on.

YOU GET

Fewer steps, and recovery

Usually fewer total calls, because it stops repeating dead ends. And a trace a human can actually read when something goes wrong.

HOLD ON TO THIS

More tokens per step, fewer steps overall. Which way that nets out depends on your task, so measure it rather than assuming. You already have the tool for that from Lecture 3.

One misconception worth killing

WHAT IT LOOKS LIKE

The agent is thinking

It writes a sentence of reasoning, then acts on it, and the reasoning looks like the cause of the action.

WHAT IS ACTUALLY HAPPENING

It is writing text that then becomes context

The sentence is generated the same way any sentence is. It helps because it lands in the transcript and conditions what comes next, not because a deliberation happened somewhere.

WHY THE DISTINCTION IS PRACTICAL, NOT PHILOSOPHICAL

If reasoning were real deliberation you could trust it as an explanation of the answer. Because it is generated text, a model can produce sound reasoning and then give an answer that does not follow from it. You will see exactly that happen at least once this semester.

SECTION 02

The other shape

Plan first, then walk it. And when that is the better bet.

Plan then execute

STEP 1	**Planner**	One model call. Given the goal and the tool names, write a numbered plan. Calls no tools.
STEP 2	**Executor**	A different prompt, with the plan in front of it. Follows the plan using the tools.
STEP 3	**Adapt**	If a step fails, the executor says so explicitly and deviates. A plan is a starting position, not a contract.

THE THING THAT MAKES THIS WORTH KNOWING

The plan exists as text before anything runs. You can log it, show it to a user for approval, diff it against what actually happened, or refuse to execute it. None of that is possible with ReAct, where the route only exists in hindsight.

Which one, and when

Route is predictable	**Plan then execute**	Cheaper. One planning call, then straight execution.
Route depends on findings	**ReAct**	A plan written before the first observation cannot use it.
A human must approve	**Plan then execute**	The plan is inspectable before anything happens.
Failures are common	**ReAct**	Recovery is built into the shape rather than bolted on.
You need an audit trail	**Either, with traces**	Both give one. Plan first gives you intent as well as actions.
Latency matters most	**ReAct, or neither**	Plan first adds a whole call before anything useful happens.

Most production systems use both: a plan for the shape, ReAct inside each step.

SECTION 03

Reflection

Checking its own work, and why that is weaker than it sounds.

The evaluator and optimizer pattern

```
result = reflect(client, MODEL, goal, draft)

# call 1: a strict reviewer lists what is missing or wrong
#         if the draft is fine it replies APPROVED and stops
# call 2: only if needed, rewrite to address every point
```

WHAT IT CATCHES WELL

Sloppiness

Missing figures, unanswered parts of the question, a date left vague, an answer that ignored half the goal.

WHAT IT CATCHES BADLY

Confident wrongness

The same model that wrote the draft is judging it. It shares the draft's blind spots, so the errors it is most sure about are the ones it will approve.

HOLD ON TO THIS

The APPROVED shortcut matters: a good draft costs one extra call, not two. Reflection you cannot afford is reflection you will switch off.

The honest limit

> **A model checking its own answer is a student marking their own exam.**

It will catch arithmetic slips and missed questions. It will not catch a misconception it holds, because the misconception is exactly what makes the wrong answer look right to it.

WHAT ACTUALLY WORKS, AND IT IS UNIT 5

Independent checks. A different model as reviewer. A deterministic validator in code. Checking the answer against a retrieved source rather than against the model's own opinion. Self critique is a cheap first filter and it is not a safety net.

SECTION 04

Three loops that do not end

And the one that is genuinely hard to see.

The three failure shapes

RUNAWAY — It never stops asking for tools

THRASHING — Same tool, same arguments, over and over

STUCK — Different calls, identical observations

ONLY THE FIRST IS CAUGHT BY WHAT YOU HAVE BUILT SO FAR

max_steps bounds all three, eventually, at full price. Detecting the second and third early is the difference between spending six calls and spending your budget before admitting defeat.

Exit 3, built

```python
@dataclass
class NoProgress:
    repeat_limit: int = 3      # same call, same args
    stuck_limit: int = 4       # different calls, same result

    def verdict(self) -> str | None:
        recent = self._calls[-self.repeat_limit:]
        if len(set(recent)) == 1:
            return f"Repeated the same call {n} times..."

        recent = self._observations[-self.stuck_limit:]
        if len(set(recent)) == 1:
            return f"Last {n} observations were identical..."
        return None
```

Two signals

They catch different bugs.

Repeat usually means the model did not register the result.

Stuck usually means the data cannot answer the question at all.

One is a model problem. The other is your problem.

It has to stay quiet when things are working

```
det.record("get_room_availability", {"hotel": "Taj Palace"},
           "0 rooms available.")
det.record("get_room_availability", {"hotel": "Radisson Blu"},
           "11 rooms available.")
det.record("get_hotel_details", {"hotel": "Radisson Blu"},
           "Rs 6200 per night.")

det.verdict()   #  None
```

A DETECTOR THAT FIRES ON HEALTHY BEHAVIOUR IS WORSE THAN NO DETECTOR

HOLD ON TO THIS

The same principle applies to every guardrail you build for the rest of this course. Test that it fires. Then test that it does not fire when it should not.

SECTION 05

Live build

Two agents, one task, and a detector that has to earn its place.

What we are doing

1	**Run act only**	Same tools, same loop, a prompt that forbids reasoning. Watch it thrash.
2	**Run ReAct**	Change only the system prompt. Watch it recover on its own.
3	**Compare**	Steps, stop reason, and answer, side by side in one table.
4	**Add the detector**	Rerun act only with NoProgress attached. Watch it stop early and say why.

OPEN THIS NOW

notebooks/u1/l4_planning.ipynb The Taj Palace is full on the target date, exactly as in Lecture 2. That is the wall both agents hit, and only one of them walks around it.

Hold on to this

Reasoning helps because it becomes context, not because deliberation happened.

The thought lands in the transcript	Next turn it reads its own intention beside the result.
Thrashing looks fine in a log	Valid calls, valid results, no errors, no progress.
Self critique shares the blind spot	Catches sloppiness. Misses confident wrongness.
Test that a guardrail stays quiet	One that fires on healthy behaviour will get switched off.

Before next session

DO	**Add reasoning to your agent**	Take your Practical 2 agent and give it a ReAct system prompt. Record steps before and after on the same question.
DO	**Attach the detector**	Wire NoProgress into your loop. Then construct a question that makes it fire, and one that must not.
THINK	**Pick a shape**	For your capstone idea, argue in three sentences for ReAct or plan then execute. Use the table, not a preference.

ON THE SECOND ONE

The question that must not fire the detector is the harder half of the exercise, and it is where the marks are. Anyone can build an alarm that goes off.

Next session

Unit 1, Lecture 5

Agent Lifecycle and Design Principles

You can now build one. Next we look at what happens around it: how an agent goes from an idea to something running, what to decide before you write any code, and the design principles that separate a system somebody maintains from one they rewrite.

COME WITH

Your ReAct comparison, your detector wired in, and your capstone shape argued in three sentences.